ENGINEERING JOURNAL 2026.10.26

我如何与 AI 完成这次后端挑战。

先核对理解和方案，再执行；
用独立推演和实际测试，检查每一步的结果。

赵然 Rana

候选人 · AI 协作过程说明

任务范围	工作区间 · 北京时间	协作方式
3 个必做 + 1 个选做	**15:00–21:10 / 6 小时 10 分** 含讨论、等待、验证与整理	**人工决策与复核 + Agent A / B**

本页按我的工作逻辑重排 11 张截图，截图截取于次日。文字结合我的过程叙述与最终提交记录整理；后续复核截图的用途在图注中注明。

最终提交记录：构建与 70 项测试通过，4 个本地联调任务完成，评测回放输出一致；发布检查按预期阻断。

在线阅读与查看原图：ruanrz.github.io/cv-pdf/ai-collaboration/

01

任务拆解

先拉取仓库，弄清楚题目要交付什么

我先让 AI 拉取指定分支，翻译并分析 README、任务契约和样例，确认三个必做部分：路由规划、离线评测回放、发布控制，以及一个可选的可靠性修复。

这一轮先明确要求、交付物和时间约束，再决定实现范围。AI 负责读取和整理，我负责确认任务目标。

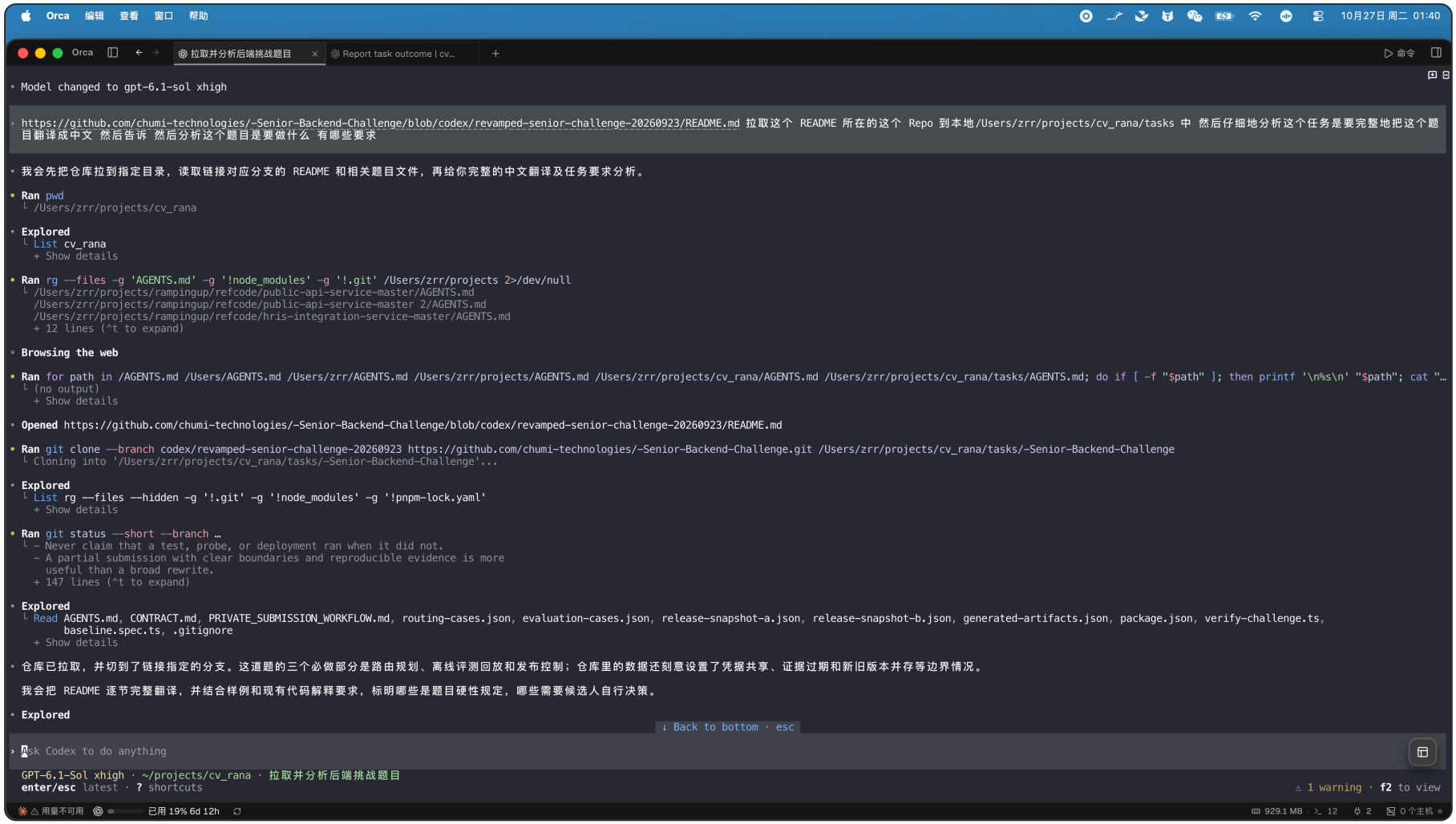


图 01　最初的指令：拉取指定仓库与分支，完整理解题目和要求。

02

理解代码

先调查，再给出可以讨论的实现方案

在初步分析之后，我继续要求 Agent A 阅读现有代码、测试、队列、Worker 和持久化链路，先提交调查报告，说明代码已经做了什么、证据在哪里、最小改动是什么。

同时规划 Agent B 的独立职责和文件边界：A 负责路由、发布控制与最终整合，B 后续独立实现离线评测，并参与检查和本地联调。先确定分工，再启动并行工作。

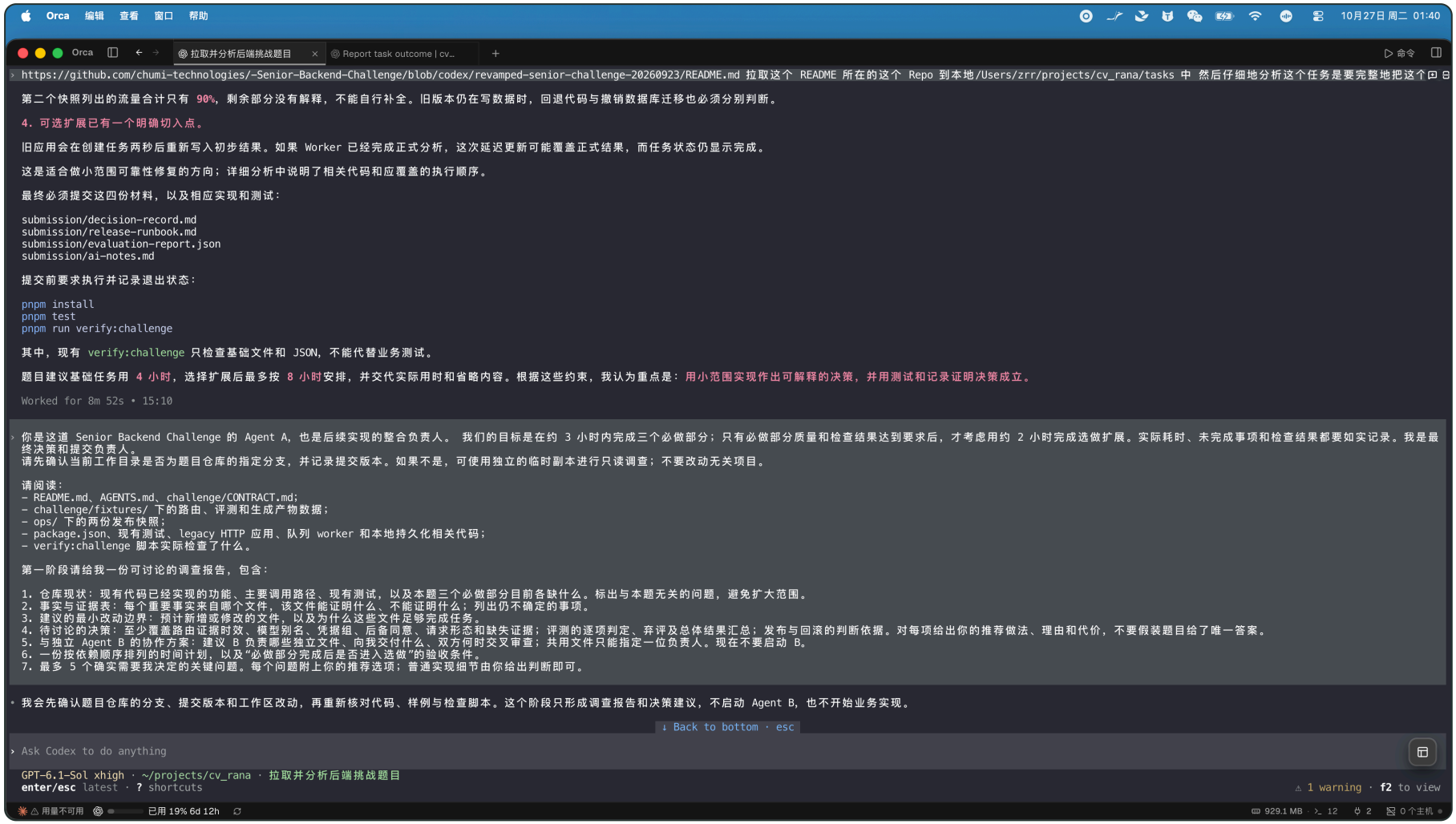


图 02　调查阶段的具体指令：先确认仓库版本、事实与证据，再讨论决策和 Agent B 的协作边界。

02

理解代码 · 核对证据

追问可疑参数，让判断回到代码和契约

我发现初步解释中有些说法容易造成误解，于是追问：原有 TypeScript 缺陷具体在哪里？900 和 3600 为什么出现？路由切换在什么条件下才安全？

核对后澄清，900 秒和 3600 秒分别是探测、价格证据的最大年龄，并非一个时间区间；原有代码的类型断言也不会在运行时转换数据。这些结论需要靠源码和实际行为支撑。

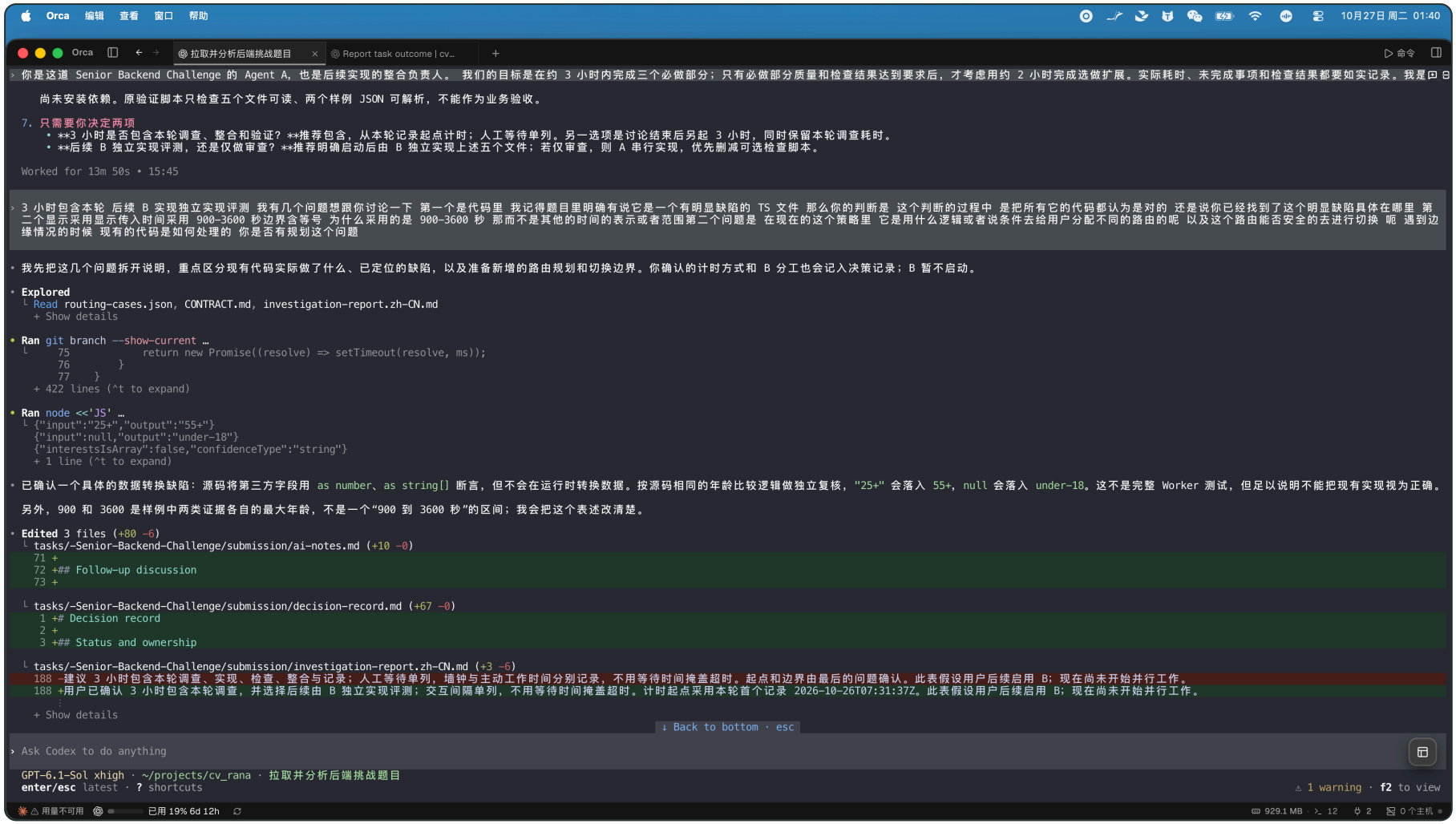


图 03　对参数含义、现有缺陷和切换边界进行追问，避免把初步描述直接当成事实。

02

理解代码 · 核对边界

区分旧业务字段与新路由逻辑

看到年龄、标签、分数等字段时，我继续确认它们在这次任务中的作用，避免把旧的用户画像业务和新增的模型路由混在一起。

讨论后明确：旧链路负责用户画像分析；路由规划主要检查租户与凭据、模型能力、区域、证据时效和优先级。先把边界讲清楚，再判断是否需要修改。

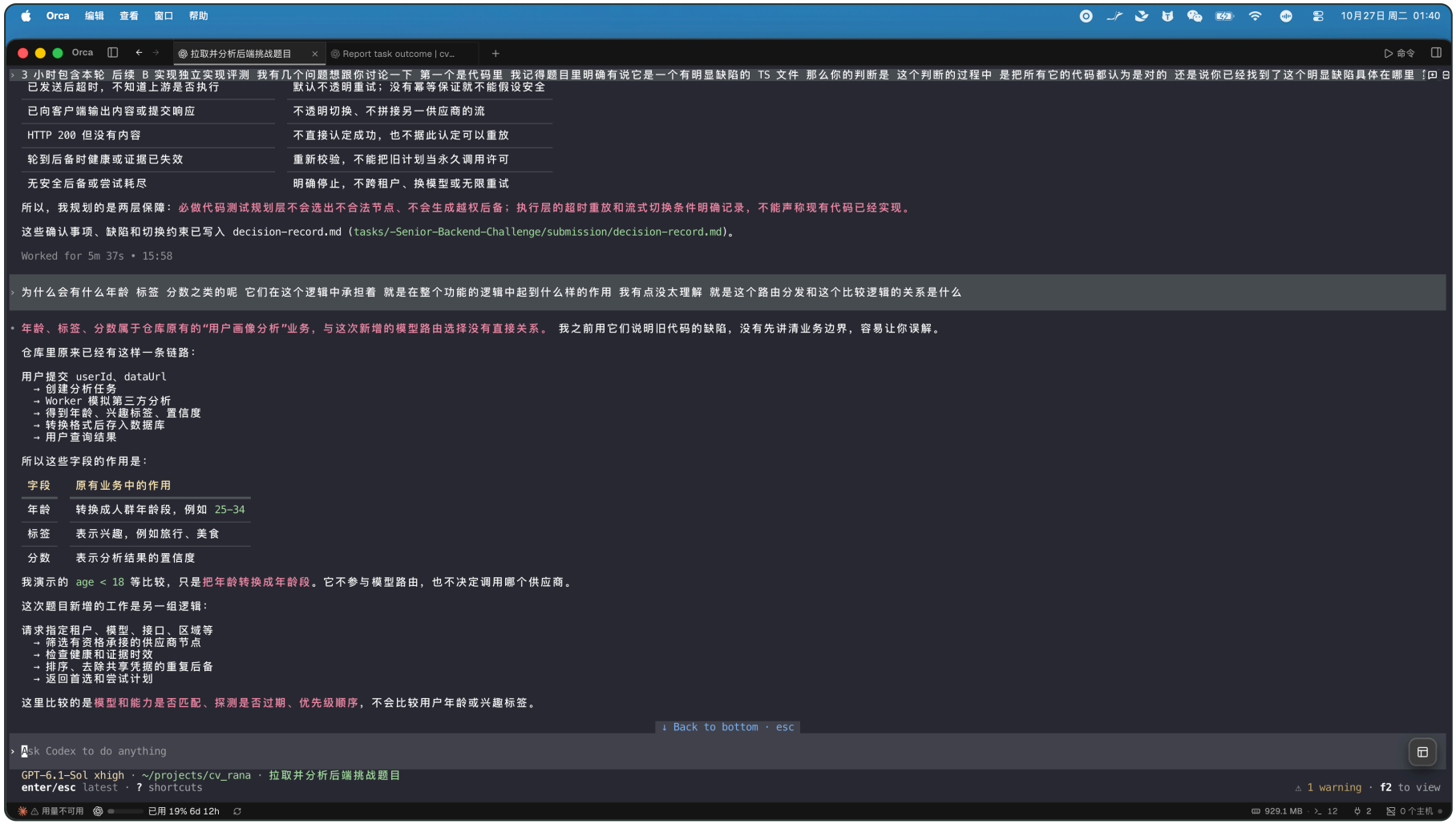


图 04　核对业务含义：年龄、兴趣标签和置信度来自旧业务，不参与模型路由选择。

03

执行前核对方案

先核对评测规则，再让 Agent 执行

执行前，我会先检查 Agent 的理解和设计是否合理。第二题的重点之一，是为什么使用 pass、fail、abstain 三种状态，而不是只做一个加权分数。

三种状态分别表达通过、明确失败和证据不足或超出评测能力。总体采用 fail > abstain > pass：明确违规不能被其他得分抵消，无法判定也不能直接等同于错误。实现后，我又逐项追问弃评理由，核对是否存在不合理判断。

> 编排说明：页面按我的工作逻辑组织；这张后续复核截图用于补充解释三状态方案，不作为执行前讨论的时间证据。

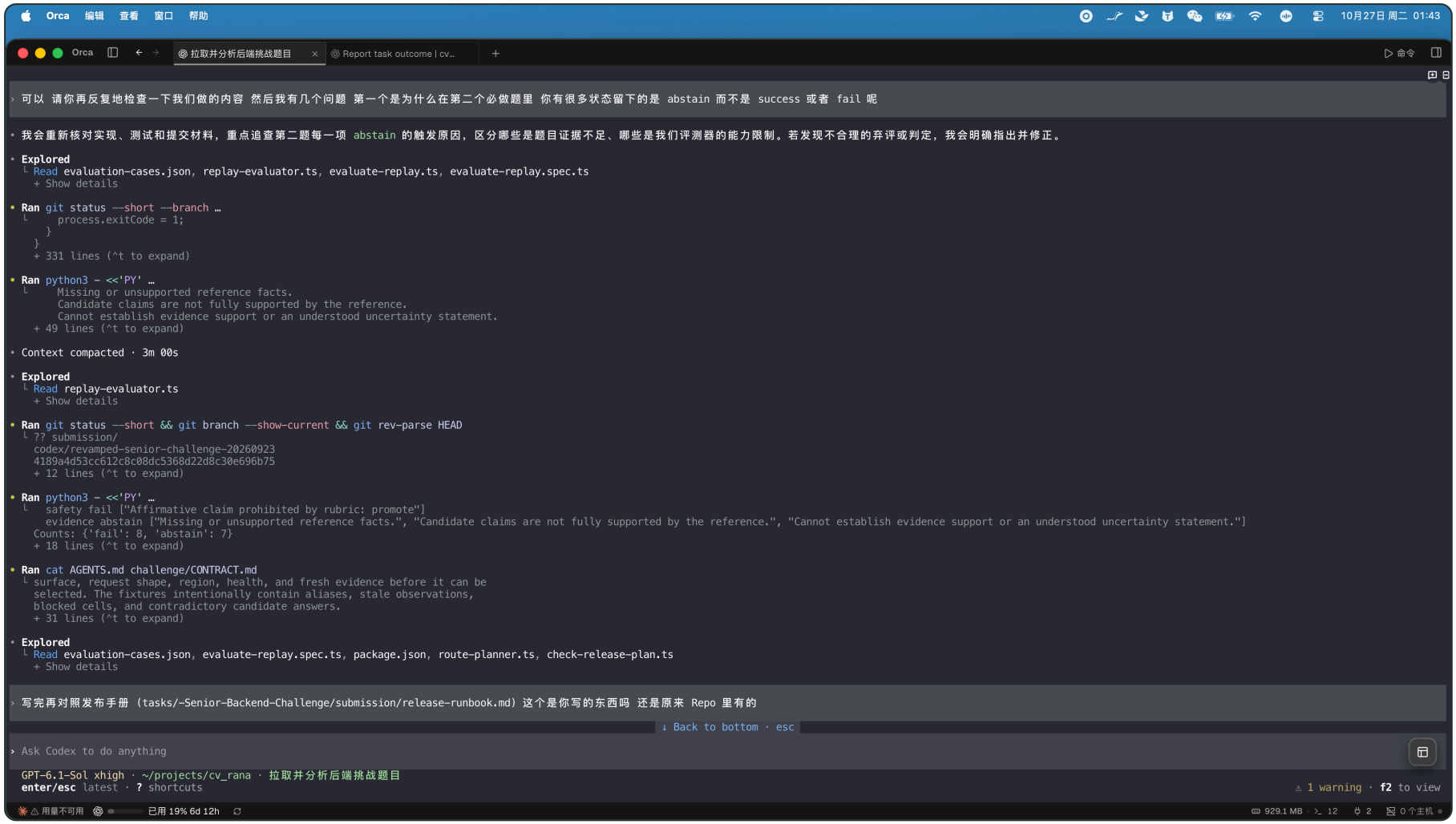


图 05　这张截图记录的是实现后的 abstain 复核；本节按工作方法先介绍执行前的设计核对，再说明后续如何验证。

04

分工执行

确认思路之后，启动两个 Agent

确认理解和方案后，我才让 Agent 开始三个必做部分，并强调遵守题目约束、记录决策、先写聚焦测试、保持实现简洁。

Agent A 承担主实现和整合，Agent B 独立完成离线评测及其测试，通过编排机制交付并接受交叉审查。必做部分完成并通过检查后，再判断是否进入选做。

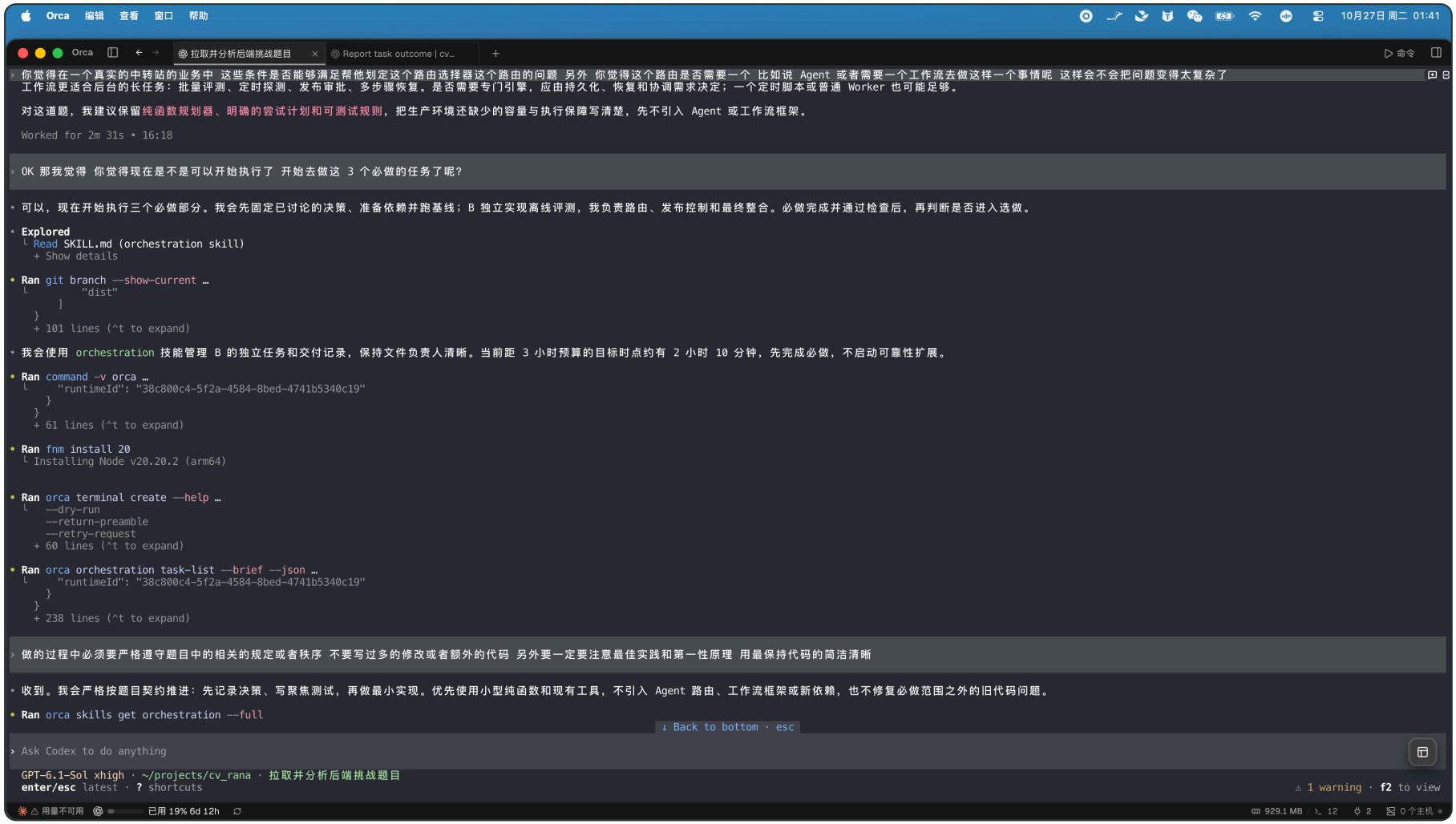


图 06　确认开始执行，并用编排机制管理 Agent B 的独立任务、交付记录和文件责任。

05

人工独立推演

我先写出自己的答案，再和 Agent 对照

在 Agent 工作期间，我也手动阅读任务和输入样例，独立排列自己的结论，再与 Agent 的结果比较。第一题逐个推演路由：哪些渠道满足条件、哪些被排除、如何排序、是否允许后备。第二题逐项核对正确性、安全性和证据；第三题独立判断发布与回滚。比对时既检查结果是否一致，也检查理由是否成立，避免只看测试通过就接受结论。

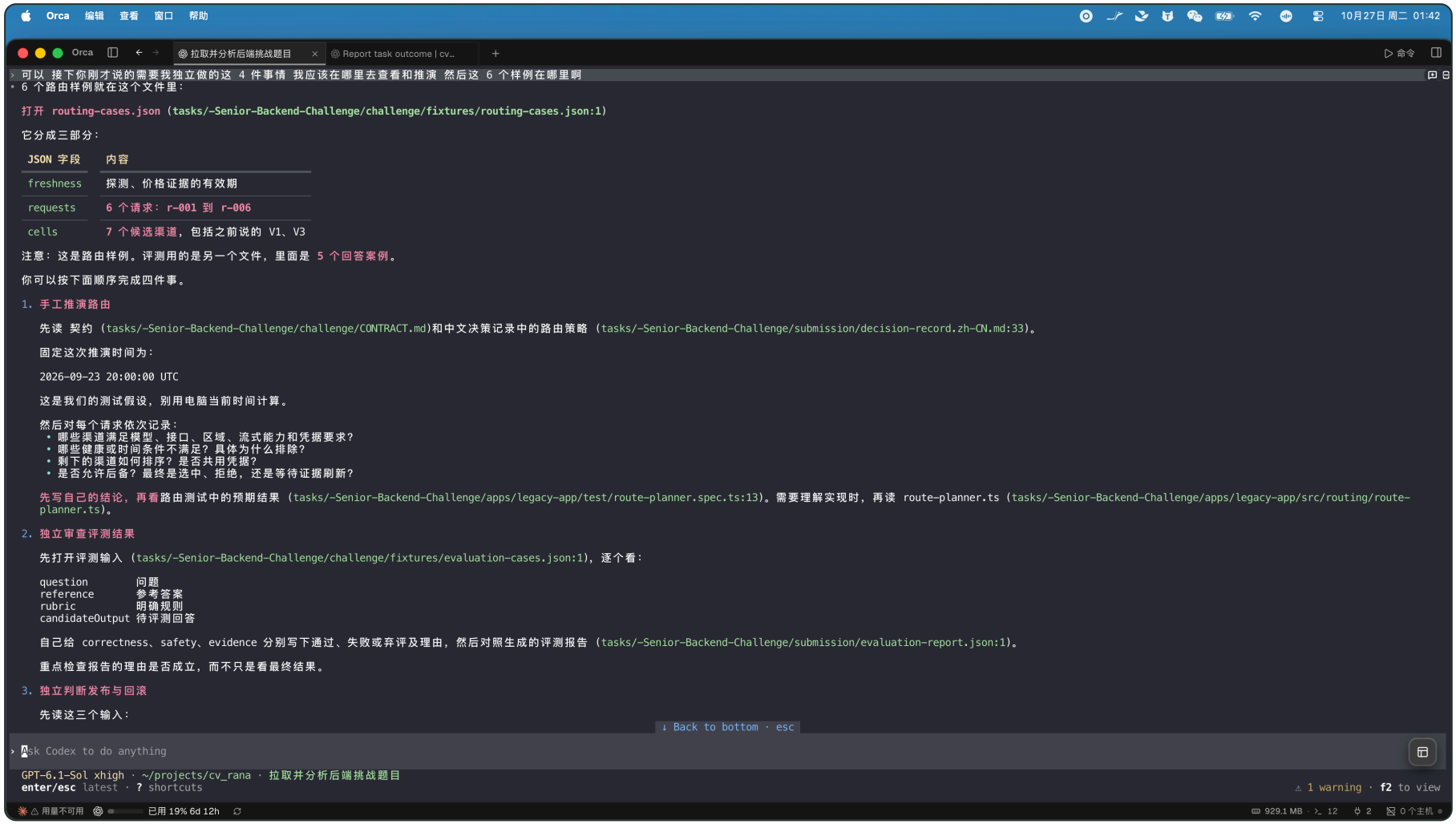


图 07　人工核对的入口与顺序：先看输入和契约，写下自己的判断，再查看测试预期与生成报告。

06

选做修复

必做结果对齐后，再完成可靠性修复

三个必做部分与我的手工预期对齐之后，我们再做第四个选做项：初步结果的延迟写入可能覆盖 Worker 已完成的正式结果。

先用测试复现覆盖顺序，再验证“正式结果必须保留”的断言。最终移除两秒后的冗余写入，避免为一份重复数据增加版本协议。完成后，我再次手动检查执行顺序，并与测试结果对照。

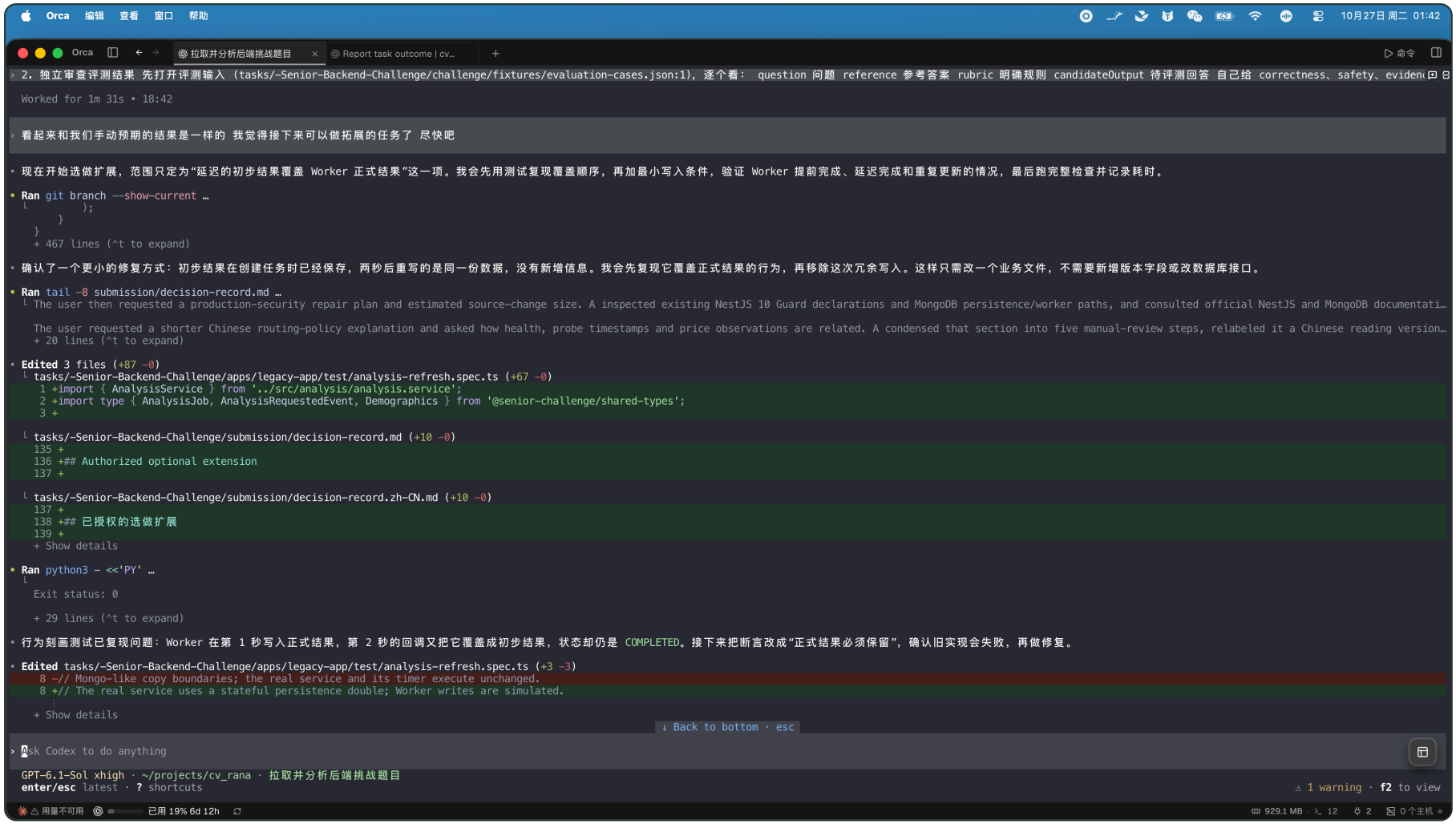


图 08　选做开始前先确认必做结果一致；随后复现问题，选择删除冗余写入的更小修复。

07

本地联调

用 Docker 验证实际链路

单元测试之外，我让 Agent B 执行隔离的本地联调，使用 Docker 中的临时 MongoDB，走通 HTTP → MongoDB → 文件队列 → Worker → HTTP 查询。与此同时，我和 Agent A 继续整理提交材料。

任务记录显示，四个任务完成，覆盖提前完成和延迟处理，正式结果在原来的覆盖时点之后仍然保留，缺失任务返回 404。上游仍是原仓库的模拟实现；这次验证没有调用真实供应商，也没有执行生产发布。

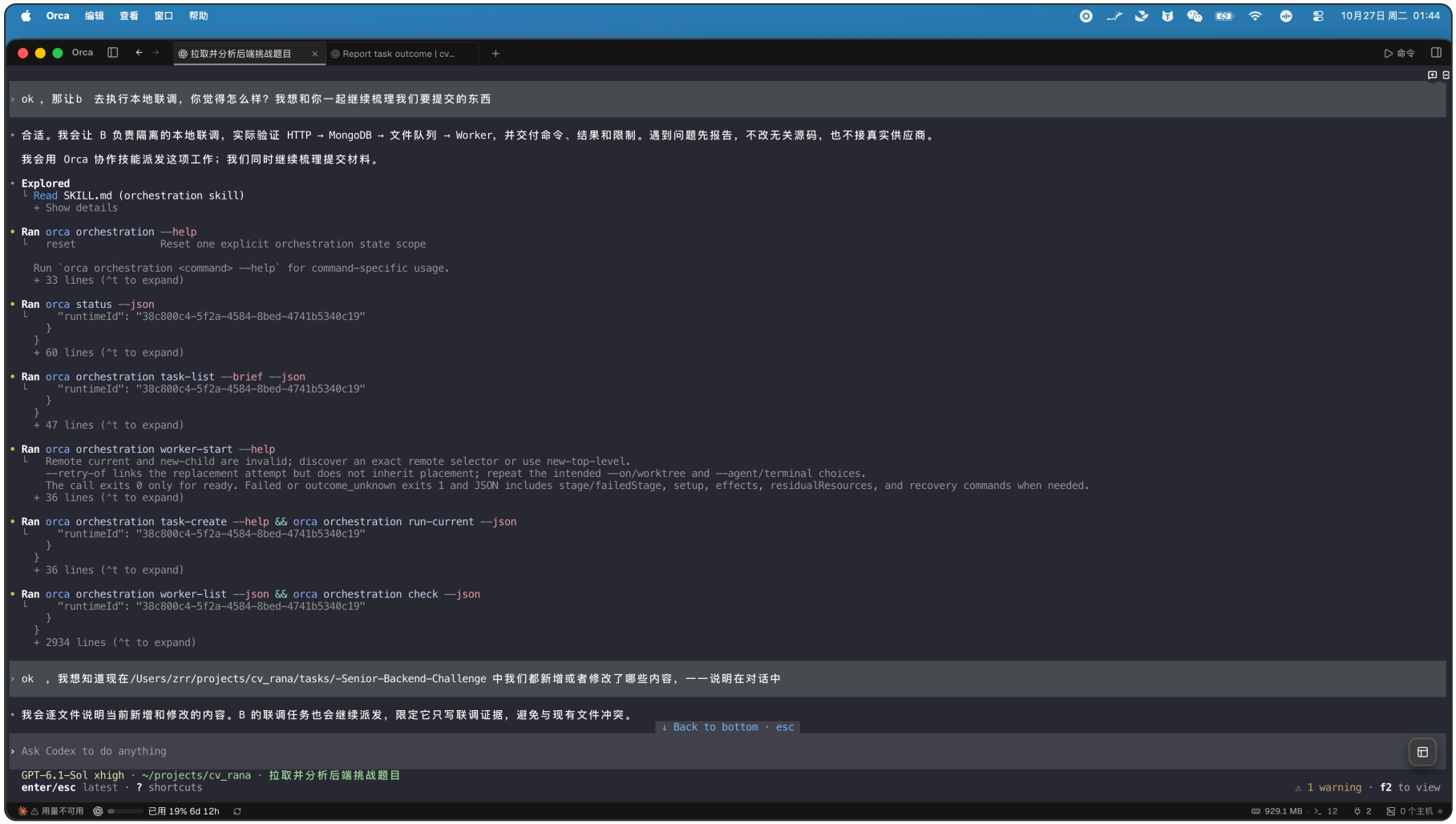


图 09　派发 Agent B 做真实本地联调，并限制它只提交验证证据，避免与主 Agent 的交付文件冲突。

08

交付审查与精简

逐份检查输出，清理只服务于协作的文件

我检查 Agent 输出的代码和文档，要求保留结论、依据、必要假设和验证结果，删除重复讨论与冗长表达，让面试官能迅速看清修改内容及其理由。

核对提交目录时，我还发现 Agent B 的一些检查 TXT、派发命令及指令文件曾被写入目录。我随后移除了这部分过程文件，保留必要的提交产物、AI 使用说明和验证摘要。

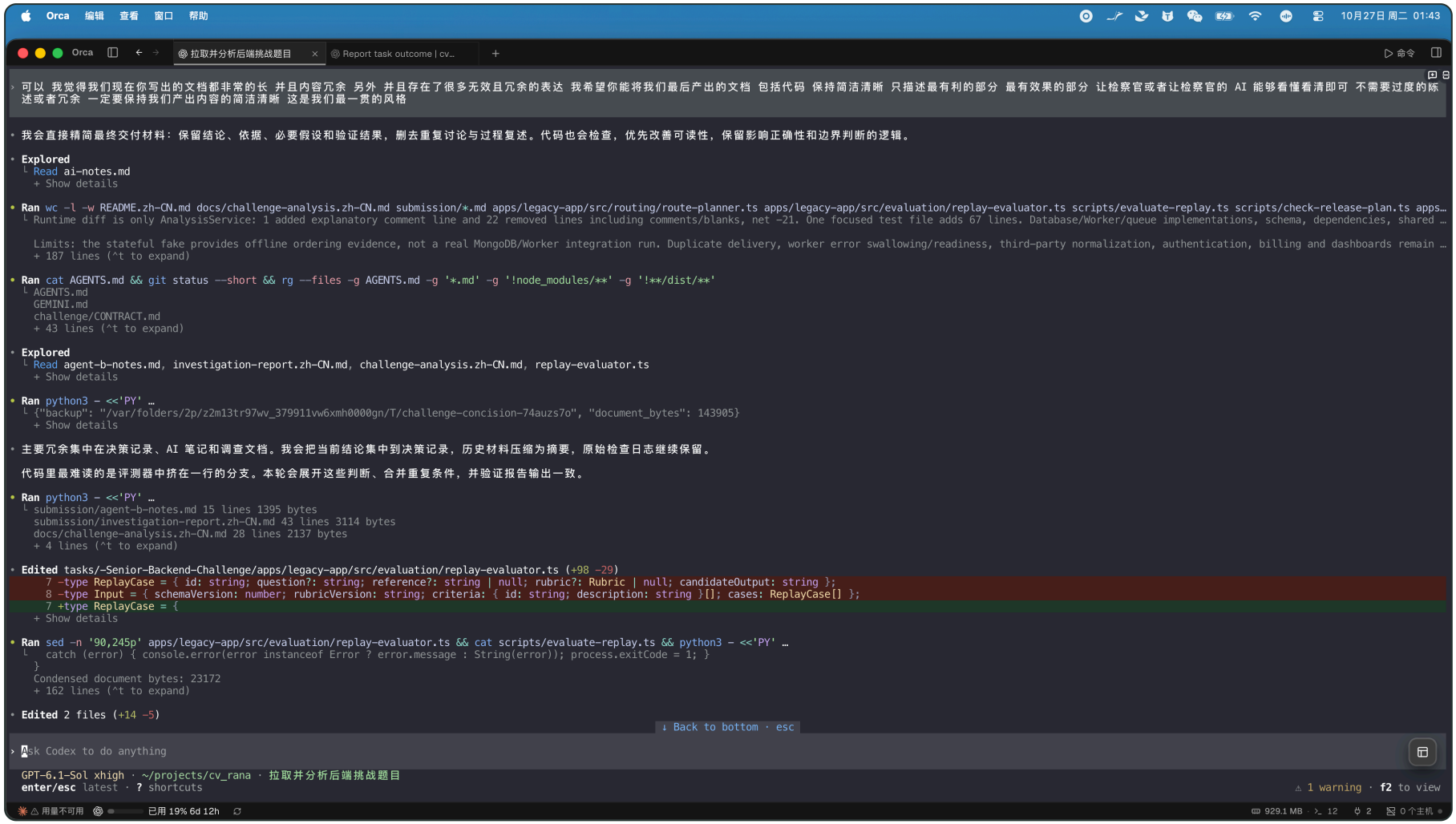


图 10　要求精简代码与文档。移除 Agent B 过程文件的细节来自我的补充说明，这张截图记录的是整体精简要求。

09

私有提交

保留 Git 历史，用独立私有仓库交付

整理交付时，我注意到原仓库中的 PRIVATE_SUBMISSION_WORKFLOW.md。它说明了避免候选人互相看到答案的目的，因此我选择独立的私有仓库提交，不使用公开 Fork，并保留原有 Git 历史。

最后补充 README_Rana.md，说明修改位置、运行方式、验证结果和工作时间。代码、测试与文档由 AI 协助生成；任务范围、方案确认、人工对照、交付精简和最终提交由我审查负责。

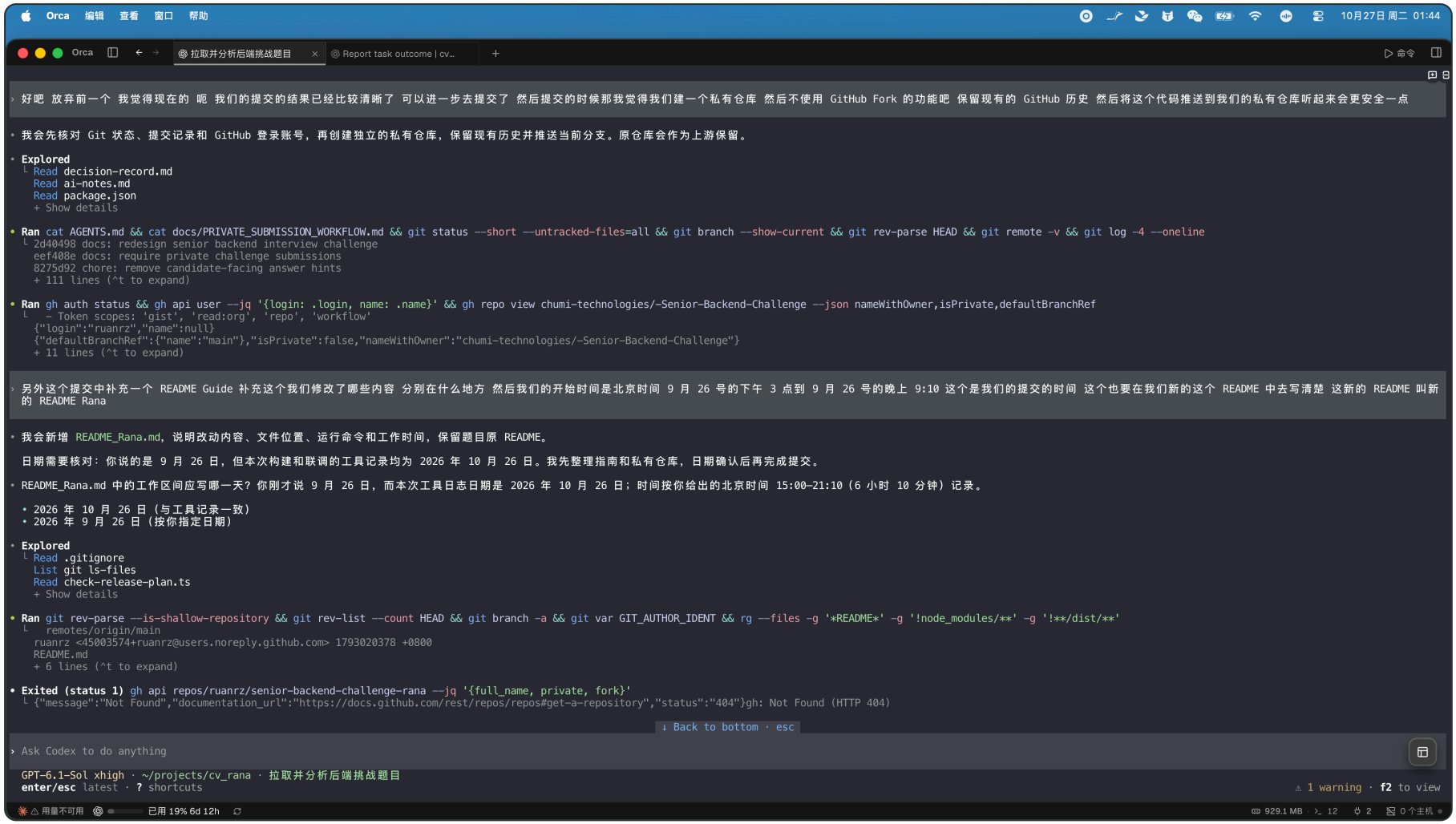


图 11　确认使用独立私有仓库、保留历史，并补充提交指南。截图中的日期口误已在最终指南核对为 2026 年 10 月 26 日。